

Governing how an agent spends

A working Cloudflare Worker, and the product thinking behind it · Nivedita Thapa

Your post on agents provisioning and paying on Cloudflare raised the obvious worry, what stops an agent from buying dozens of domains, and answered it with a flat \$100 per month cap. I built the layer that sits where that cap is, and the first thing I learned is that a single number cannot tell the difference between the purchases that are fine to make automatically and the ones that should stop and ask.

What it is

A Cloudflare Worker, deployed and running, that decides in real time whether an agent gets to make a given purchase. Every action the agent attempts returns one of three verdicts, each with a logged reason:

- **approve:** when the charge is within budget and within an auto-approve rule. The agent proceeds, no human needed.
- **hold:** when the charge is affordable but consequential. It waits for a one-tap human yes while the agent keeps working on everything else.
- **deny:** when the charge would break the budget.

```
agent goal: build and ship an app on Cloudflare
```

```
[APPROVE] Create R2 bucket for assets (free)
Free service, no spend impact.
$0.00 spent, $100.00 left
```

```
[APPROVE] Register coolapp.dev ($10.44)
Within the $20 auto-approve threshold.
$10.44 spent, $89.56 left
```

```
[HOLD] Workers Paid subscription ($5.00)
Recurring charge, held for human approval.
$10.44 spent, $89.56 left
```

```
[HOLD] Register coolapp.ai ($70.00)
Over the $20 threshold, held for approval.
$10.44 spent, $89.56 left
```

```
[HOLD] Register coolapp.io ($34.00)
Over the $20 threshold, held for approval.
$10.44 spent, $89.56 left
```

The agent working a real purchase queue against the deployed Worker.

Budget and a full audit trail persist in Workers KV. The decision logic lives in its own tested module, so the judgment is inspectable, not buried in plumbing. I stubbed the Stripe identity and payment-token exchange on purpose: the protocol already handles those, and rebuilding them would have added wiring without

adding insight. The interesting problem is the governance, so that is the only thing I built.

The insight a cap misses

- A one-time \$12 domain is done the moment it is bought. A \$12 per month subscription is a standing liability that keeps charging long after the agent forgot it existed. The bills people get burned by are almost never the big one-time purchase, they are the small recurring ones nobody is watching. So recurring charges always stop for a human, regardless of how small.
- Registering a domain is hard to undo. Spinning up a bucket is trivial to tear down. How easily an action can be reversed is a better reason to pause than its price, and the catalog does not expose that signal today.
- A \$15 charge with \$85 left is routine. The same charge with \$8 left is the one that strands the agent halfway through its task. The decision has to read the budget that remains, not just the sticker price in front of it.

Product feedback, in priority order

- **Make the catalog carry charge semantics:** To govern well I needed to know whether a charge recurs and how reversible it is. Today both have to be guessed. If Discovery returned that structure, every governance layer built on top, yours or a partner's, gets better for free, and agents themselves make smarter calls.
- **Ship a hold:** A cap passes everything underneath it silently and kills everything above it. The primitive that actually keeps a human in control without making them babysit is the middle state: let the agent keep moving while one specific charge waits for approval. That is much closer to the human-in-the-loop idea the post describes than a hard ceiling.
- **Treat the audit trail as a product:** The day agents spend on their own, every customer asks the same question: what did it buy, and why was each one allowed. The decision-and-reason record I built for governance is what a human gets to read afterward.

Live: agent-spend-governance.niveditathapa.workers.dev/catalog · Code, tests, and a from-scratch deploy guide are in the repo.